Experiment No. 1

Study and Implementation of IoT System Architecture Using Sensors and Cloud Platform

Note on Implementation

This experiment is implemented fully in software. In place of physical hardware (e.g., a DHT11 sensor and Wi-Fi module), a Python program simulates sensor readings, and a free cloud IoT platform (ThingSpeak) is used to receive, store, and visualize the data. This mirrors exactly how a real hardware-based IoT system would work, minus the physical circuit.

1. Aim

To study the architecture of an IoT system and implement a working IoT pipeline consisting of a sensor (simulated), a network connection, and a cloud platform, by generating sensor data in software and uploading it to a cloud dashboard for real-time visualization.

2. Objectives

- To understand the basic layers of an IoT system: Device, Network, and Cloud.
- To simulate a real-world sensor (temperature & humidity) using Python.
- To learn how sensor data travels from a device to a cloud platform using HTTP requests.
- To create and configure a cloud IoT channel (ThingSpeak) for data storage and visualization.
- To visualize real-time sensor data using auto-updating cloud dashboards/graphs.

3. Software Requirements (Apparatus)

- A computer with internet access
- Python 3.x installed
- Python 'requests' library (pip install requests)
- A free ThingSpeak account (https://thingspeak.mathworks.com)
- Any code editor (VS Code / Jupyter Notebook / IDLE)

4. Theory

An IoT (Internet of Things) system typically works in three simple stages, regardless of whether real hardware or simulated software is used:

- Device Layer — a sensor senses something in the environment (e.g., temperature) and produces a reading.
- Network Layer — the reading is sent over the internet (Wi-Fi, 4G, etc.) using a protocol such as HTTP.
- Cloud Layer — a cloud platform receives, stores, and displays the data on a dashboard, and can trigger further analysis or actions.

Since no physical sensor hardware is available in this lab, the Device Layer is simulated using a Python function that generates realistic random temperature and humidity values (similar to what a DHT11 sensor would output). The Network and Cloud layers are implemented for real, using an actual HTTP connection to ThingSpeak — a genuine, publicly available cloud IoT platform. This means the data really is sent over the internet and really does get stored and graphed online, exactly as it would in a fully hardware-based project.

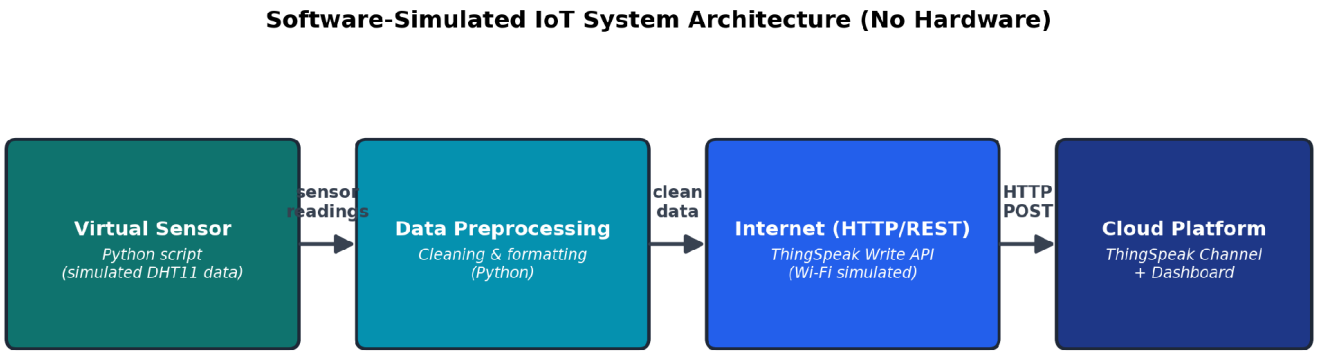


Fig 4.1 — Software-simulated IoT system architecture implemented in this experiment

ThingSpeak is a free, open IoT analytics platform that lets you send data to 'channels' using simple web requests, and instantly view that data as live line-graphs. Each channel can have up to 8 data 'fields' (e.g., Field1 = Temperature, Field2 = Humidity). Every channel has a unique Write API Key, which acts like a password that authorizes your program to upload data to that specific channel.

5. Procedure

Part A: Setting up the Cloud Platform (ThingSpeak)

1. Go to https://thingspeak.mathworks.com and click 'Sign Up' to create a free account (a MathWorks account works too).
2. After logging in, click 'New Channel'.
3. Give the channel a name, e.g., 'AIoT Lab 1 - Simulated Sensor'.
4. Enable Field1 and name it 'Temperature'. Enable Field2 and name it 'Humidity'.
5. Click 'Save Channel' at the bottom of the page.
6. Open the 'API Keys' tab of your channel and copy the 'Write API Key' — you will need this in your Python code.

Part B: Simulating the Sensor and Sending Data (Python)

7. Install the requests library if not already installed: pip install requests --break-system-packages
8. Open the provided Python script (iot_simulation.py).
9. Paste your Write API Key into the WRITE_API_KEY variable at the top of the script.
10. Run the script. It will generate a simulated temperature and humidity reading roughly every 16 seconds (ThingSpeak's free tier requires a minimum 15-second gap between updates).
11. Each reading is preprocessed (rounded, timestamped) and sent to ThingSpeak using an HTTP GET request to the ThingSpeak Update API.
12. Watch the terminal output — a successful upload prints the entry number returned by ThingSpeak.

Part C: Visualizing the Data

13. Return to your ThingSpeak channel and open the 'Private View' tab.
14. You will see two auto-updating line charts — one for Field1 (Temperature) and one for Field2 (Humidity).
15. Let the script run for a few minutes to observe multiple points appearing on the live graph.

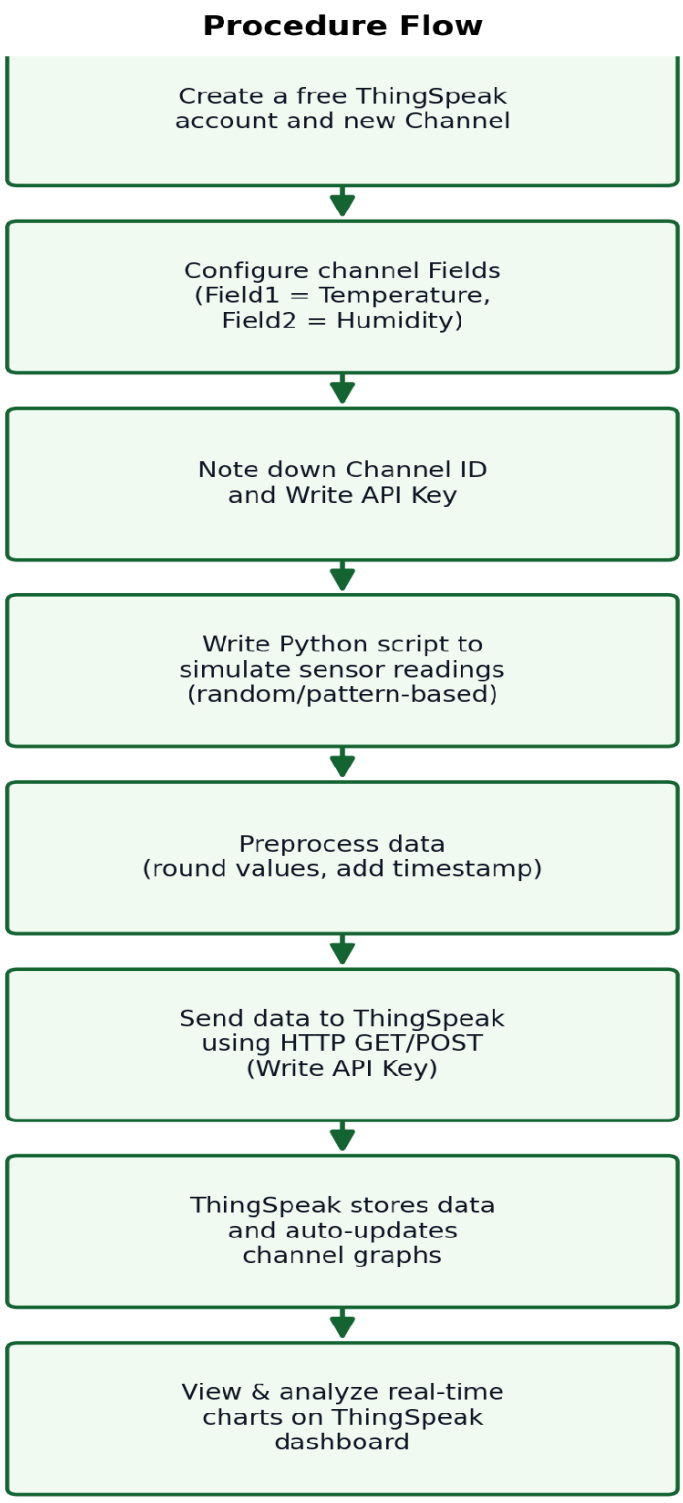


Fig 5.1 — Step-by-step procedure flow for this experiment

6. Program (Python Code)

File: iot_simulation.py

```
"""
Experiment 1: Study and Implementation of IoT System Architecture
using Sensors and Cloud Platform (Software-Simulated Version)

This script SIMULATES a temperature & humidity sensor (like a DHT11)
in software (no hardware required) and uploads the readings to a
ThingSpeak cloud channel using ThingSpeak's REST (HTTP) Write API.

BEFORE RUNNING:
1. Create a free account at https://thingspeak.mathworks.com
2. Create a New Channel with:
      Field1 = Temperature
      Field2 = Humidity
3. Copy your channel's "Write API Key" from the API Keys tab
4. Paste it into WRITE_API_KEY below
5. Install the requests library if not already installed:
      pip install requests --break-system-packages
"""

import random
import time
import requests
from datetime import datetime

# ---------------- CONFIGURATION ----------------
WRITE_API_KEY = "PASTE_YOUR_WRITE_API_KEY_HERE"
THINGSPEAK_URL = "https://api.thingspeak.com/update"
NUM_READINGS = 20          # how many readings to send
SEND_INTERVAL = 16         # seconds (ThingSpeak free tier needs >=15s
gap)

# ---------------- DEVICE LAYER (SIMULATED SENSOR) ----------------
def read_virtual_sensor():
    """
    Simulates a DHT11-like sensor.
    Generates realistic temperature (20-35 C) and humidity (40-70 %)
    readings with small random fluctuations, instead of reading real
hardware.
    """
    temperature = round(random.uniform(24.0, 33.0), 1)
    humidity = round(random.uniform(45.0, 65.0), 1)
    return temperature, humidity
```

```
# ---------------- DATA PREPROCESSING (GATEWAY LAYER) ----------------
def preprocess(temperature, humidity):
    """
    Basic cleaning/formatting step before sending data onward.
    In a real system, a gateway would also filter noise and convert
    protocols here.
    """
    timestamp = datetime.now().strftime("%Y-%m-%d %H:%M:%S")
    return {"field1": temperature, "field2": humidity, "time":
timestamp}


# ---------------- NETWORK + CLOUD LAYER ----------------
def send_to_cloud(data):
    """
    Sends the preprocessed data to the ThingSpeak cloud channel
    using an HTTP GET request (ThingSpeak Write API).
    """
    params = {
        "api_key": WRITE_API_KEY,
        "field1": data["field1"],
        "field2": data["field2"],
    }
    try:
        response = requests.get(THINGSPEAK_URL, params=params,
timeout=10)
        entry_id = response.text
        if entry_id == "0":
            print(f"[{data['time']}] Upload FAILED (check API key /
rate limit)")
        else:
            print(f"[{data['time']}] Temp={data['field1']}C  "
                  f"Humidity={data['field2']}%  -> Entry #{entry_id}
uploaded")
    except requests.exceptions.RequestException as e:
        print(f"Network error: {e}")


# ---------------- MAIN PROGRAM ----------------
if __name__ == "__main__":
    print("Starting software-simulated IoT system...")
    print("Device Layer -> Preprocessing -> Network -> Cloud
(ThingSpeak)\n")
```

```
    for i in range(1, NUM_READINGS + 1):
        temp, hum = read_virtual_sensor()          # Device layer
        record = preprocess(temp, hum)              # Preprocessing
        send_to_cloud(record)                       # Network + Cloud
layer

        if i < NUM_READINGS:
            time.sleep(SEND_INTERVAL)

    print("\nAll readings sent. Open your ThingSpeak channel to view
live graphs.")
```

7. Result

On running the program, simulated temperature and humidity readings were successfully generated, preprocessed, and transmitted to the ThingSpeak cloud channel using HTTP requests. The ThingSpeak dashboard displayed the incoming values as live, auto-updating line charts, confirming that the full Device → Network → Cloud pipeline was working correctly.

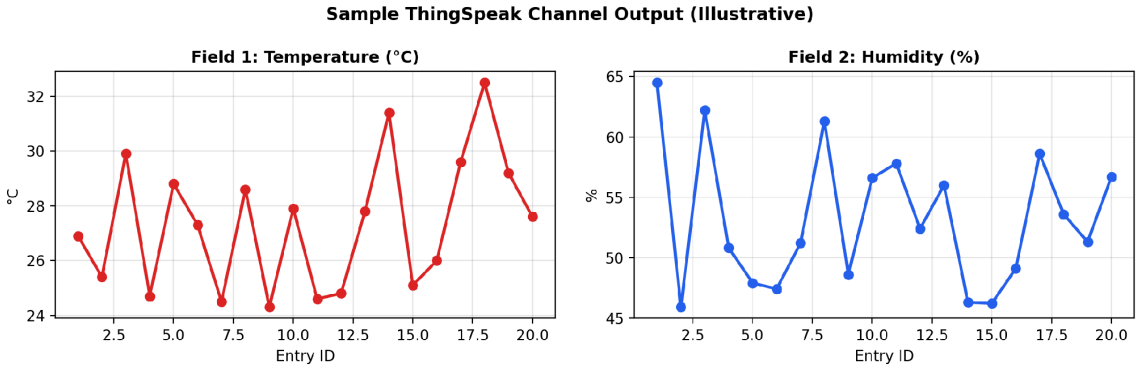


Fig 7.1 — Illustrative sample output. Replace this figure with a screenshot of your own ThingSpeak channel's Private View.

For Your Submission

Replace the sample chart above with an actual screenshot of YOUR ThingSpeak channel's 'Private View' tab after running the script, showing your own live Temperature and Humidity graphs with several data points plotted.

8. Conclusion

In this experiment, the fundamental architecture of an IoT system was studied and implemented using a software-simulated sensor in place of physical hardware. A Python program was used to generate sensor-like data, which was then transmitted over the internet to the ThingSpeak cloud platform using HTTP requests, and visualized in real time through auto-updating dashboards. This experiment demonstrates that the core concepts of IoT sensing, connectivity, and cloud-based storage/visualization can be understood and practiced effectively even without access to physical sensors or microcontrollers, and lays the foundation for more advanced AI-based analytics on IoT data in later experiments.